

Metadata-Augmented Recursive Decomposition for Cost-Efficient Comprehension of Unbounded Documents

Anugrah Shetty^{1, a, b, *}

Arav Sharma^{2, a, b}

Parth Sangani^{3, a, b}

Tanish Sharma^{4, a, b}

Soni Sweta^{5, a, b, *}

Prashasti Kanikar^{6, a, b}

^aDepartment of Computer Engineering

^bMukesh Patel School of Technology Management & Engineering, SVKM's Narsee Monjee
Institute of Management Studies (NMIMS) Deemed-to-University, Mumbai-400056, India

¹anugrahsshetty@gmail.com

²arav.sharma0405@gmail.com

³sanganiparth22@gmail.com

⁴tanishsharma619@gmail.com

⁵soni.sweta16@gmail.com

⁶Prashasti.Kanikar@nmims.edu

Corresponding Authors*

Abstract—Recursive Language Models (RLMs) process inputs far beyond a model’s native context window by holding the document in a REPL environment and letting a root model explore it programmatically, delegating reading to sub-model calls. We observe that this exploration is structurally blind: each recursive call receives a raw text slice with no view of the document’s organisation, no record of what sibling calls have already established, and no statement of why it was invoked. On documents whose structure is exploitable—textbooks, manuals, standards—this discards precisely the information that makes the document navigable. We propose Metadata-Augmented Recursive Decomposition (MARD), an inference-time extension that threads a metadata envelope through every recursive call. The envelope carries a structural skeleton derived from the document’s own text, findings accumulated from prior calls, and a parent-issued directive, and it grows as exploration proceeds. Exploration thereby becomes an act of confirming a hypothesis rather than of discovering blindly. We instantiate MARD on dependency-ordered study-material generation over textbook-scale documents, where a frontier “Scout” reads a small fraction of the document to emit a typed Master Plan and a fleet of cheap “Swarm” builders generate material that is joined in dependency order rather than book order. We evaluate MARD against vanilla RLM under matched models, matched recursion depth and identical structural controls, measuring output quality and tokens consumed independently, with three repeated runs per condition and a flat-context negative control. We additionally instrument the *groundedness* of generated content, after observing the baseline silently produce explanations without consulting the source document while reporting success. On explanation coverage the two arms are indistinguishable—MARD 0.621 against the baseline’s 0.551, over ranges that overlap—and we report that as a null. The contribution is elsewhere. MARD consumes $15.8\times$ fewer input tokens, and it is stable where the baseline is not: across three identical repeats the baseline’s coverage ranges from 0.315 to 0.765 and its output organisation from 75 to 190 sections, while MARD’s range 0.584–

0.650 and 83–84 concepts, each carrying a source span that resolves against the document; the baseline emits no resolvable citation in any repeat. Ablating the envelope by channel isolates the mechanism: removing the accumulated findings collapses cross-chapter dependency linkage from 0.917 to 0.014, while removing the structural skeleton—the most expensive component of the frontier tier’s input—changes nothing we can measure. On a structure-ablated document MARD declines to run at zero cost, whereas the baseline produces confident study guides that acknowledge no disorder. We report two limitations that bound these claims: MARD’s builders never receive the cited prose, so its explanations are attributable but not text-grounded, and we did not reproduce a published baseline result first-hand.

Index Terms—long-context language models, recursive inference, document structure, prerequisite ordering, inference-time methods, cost-efficient inference

1. INTRODUCTION

The dominant approach to reasoning over documents that exceed a language model’s context window is to reduce the document: chunk and retrieve, compress the prompt, or compact the conversation. Each discards information and each does so before the model has had a chance to decide what matters.

Recursive Language Models [1] invert this. The document is never placed in the prompt at all. It is bound to a variable inside a Python REPL, and the root model receives only metadata about it—its type, its length. The root then writes code to explore it: slicing, regular expressions, loops. Inside that REPL it may call `llm_query`, handing an extracted slice to a sub-model that reads it and returns an answer. The root is an orchestrator that decides what to read and delegates the reading. Because the root sees only truncated program output,

its own context grows slowly, and inputs of several million tokens become tractable against a context window two orders of magnitude smaller.

1.1. The defect: recursion without orientation

We observe a specific deficiency in this design. Every sub-call is given a raw slice of text and nothing else. It does not know which document the slice comes from, where in that document it sits, what other sub-calls have already established, or why it in particular was invoked. It is, in effect, a reader handed three photocopied pages with no cover and no page numbers.

Two costs follow. First, orientation is re-derived on every call rather than established once and reused. Second, sub-calls cannot build on one another: a finding established in Chapter 3 is unavailable to the call that reads Chapter 9, even though the document itself may explicitly connect them. On a flat context—a shuffled transcript, a log file—neither cost is avoidable, because there is no structure to be oriented against. But on a document with exploitable structure, the information being discarded is exactly the information the author supplied to make the text navigable: a table of contents, numbered sections, and in-text cross-references of the form “as we saw in Chapter 3.”

1.2. Contribution

We introduce a *metadata envelope*: a compact, growing structure carried into every recursive call, comprising (i) a structural skeleton of the document, (ii) findings accumulated across prior calls, and (iii) a directive issued by the calling context stating what this call is for. Our claim, stated in the form we commit to before reporting any measurement:

MARD’s growing metadata envelope makes recursive document exploration structure-aware—improving output quality and/or reducing tokens consumed relative to vanilla RLM on documents with exploitable structure, while degrading gracefully to vanilla-RLM behaviour on documents without it.

Three properties of this sentence are deliberate. It credits the envelope, not recursion, because recursion is the base paper’s. It is a disjunction over quality and tokens, because we do not know in advance which axis the effect appears on and measure both independently. And the graceful-degeneration clause is a prediction stated in advance, not a failure mode discovered afterwards; it is an objective of this work rather than a caveat on it.

A second property emerged from measurement rather than design, and we report it as a finding rather than folding it into the claim above. Because MARD’s plan is a *typed contract* validated at the tier boundary, a builder receives its source span by construction rather than by a model matching text to a concept at generation time. We observed the vanilla baseline silently generate content without consulting the document at all, while reporting success (Section 4.4). The envelope’s structural apparatus makes that failure mode loud rather than

silent, which is a claim about *faithfulness and auditability* distinct from output quality.

1.3. What is not claimed

It is important to say this before a reader infers otherwise. The base paper’s RLM already pairs a frontier root model with cheaper sub-model calls, and already recurses. Neither is our contribution. The base library that accompanies it already carries rich per-call metadata and even ships an example demonstrating metadata captured at every depth of the recursion tree. Section 3.1.1 draws the line precisely: that metadata flows upward and is observational, whereas MARD’s envelope flows downward and is operative. Same word, opposite direction, and only one of the two changes what the model does.

We also make no claim to beat the state of the art on long-context benchmarks. Isolating MARD’s contribution means comparing against vanilla RLM; the base paper’s own results report a higher overall high-water mark from a different system, and that remains the standing bar [1].

1.4. Demonstrating application

We instantiate MARD on the generation of dependency-ordered study material from textbook-scale documents. This application is chosen because it makes the envelope’s contribution measurable rather than anecdotal: a textbook declares its own prerequisite relations through in-text cross-references, so a generated ordering can be scored against ground truth extracted programmatically from the document itself, with no expert annotation and therefore no human-subjects component.

The application also traces a specific engineering path, and naming it makes the design constraints legible. A structured view of a document can be obtained directly, by handing the source to a long-context model and rendering the structure it emits; this is the shape of the tools in Fig. 1(a) and (b). That approach is bounded by the context window, which a textbook exceeds. Recursive Language Models remove the bound by never placing the document in a prompt—but our measurements show that what they return in its place is unstable. Across three repeats of an identical configuration the baseline’s output organisation ranged from 75 to 190 sections (Section 4.2), and given a document whose structure had been destroyed it produced three confident study guides that named the intact book’s opening section and acknowledged nothing (Section 4.6). A structure extracted once, by hand, can be inspected before it is trusted; a structure extracted on every run, varying this much, cannot.

MARD is the attempt to keep the recursion and recover the stability. What it produces is not the study text itself but a *concept graph*: concepts carrying verified source sections and page ranges, joined by prerequisite edges. That artefact is the object we intend to be rendered and navigated—an ordering over a book that reflects what must be understood before what, rather than the order in which the pages happen to be bound.

We are explicit that this manuscript does not evaluate that end. No learner used the artefact, no comparison against linear

reading was run, and no claim about learning outcomes is made or implied anywhere in this paper. What is evaluated is whether the artefact can be produced reliably, cheaply and with checkable provenance, which is the precondition for asking the learning question at all.

2. LITERATURE REVIEW

2.1. Recursive and long-context inference

Our work extends RLM [1] directly. A recent alternative framework, SRLM [2], reports up to 22% over RLM under a matched time budget and argues that recursion may not be the primary driver of RLM’s advantage, observing that within-window recursion sometimes degrades performance. We take no position on that dispute, and our design does not require one: the envelope reduces orientation cost, which is a claim about what each call is *given* rather than about what makes recursion work. The critique does, however, shape our methodology. If the objection to a reported improvement is that it may be sampling noise or prompt luck, the defence is not more documents but variance across repeated runs, a negative control, and a boundary condition stated in advance. Section 3.2 adopts all three.

Positional-degradation results [3] motivate the broader family of context-reduction approaches, of which prompt compression [4]–[6] and its recent survey [7] are representative. These are complementary rather than competing: they reduce what a call must read, whereas MARD changes what a call knows about what it reads.

2.2. Hierarchical decomposition of long documents

Hierarchical merging for summarisation [8] and multi-agent narrative summarisation [9] decompose long inputs into a tree and combine partial results upward. The structural difference from MARD is the direction of information flow: in merging schemes, context is assembled on the way up. In MARD, accumulated context is injected on the way down, before each unit of work executes.

2.3. Tiered and cost-aware inference

Routing and cascading approaches [10], [11] and hint-based delegation [12] allocate a frontier model selectively to control cost. MARD’s Scout/Swarm split is an instance of the same economics, and we claim no novelty for it. What is ours is that the cheap tier receives a typed plan directive rather than an undifferentiated slice—an ablation we isolate directly (Section 4.5).

2.4. Prerequisite structure and educational applications

Concept prerequisite extraction has an established literature [13]–[15], as do LLM agents in education [16], [18]. The application context these serve—adaptive and personalised e-learning, and the educational data mining that supports it—is surveyed in [17]. We differ in the source of supervision: rather than learning prerequisite relations from labelled corpora, we treat each edge our system emits as a falsifiable prediction against the document’s own declared cross-references.

2.5. Positioning against shipped systems

Two acknowledgements, which we state rather than let a reader supply. Ingestion capacity is not the gap—deployed products already accept large document collections—and at least one shipped product now generates sequenced study plans from uploaded course material. The line we draw is on the *source* of the sequencing: deployed systems that sequence do so diagnostically, ordering by what a learner has answered incorrectly, whereas our ordering is document-derived, extracted without supervision and scored against the text’s own cross-references. We deliberately do not use any closed product as a quantitative baseline: their architectures are unpublished, they expose no token accounting, and they change without notice, so no such comparison would reproduce. No peer-reviewed source establishes what any particular shipped product’s mind-map view does or does not encode, so we treat this as a qualitative observation supported by first-hand inspection rather than as a citable claim. Fig. 1 places three renderings of the same source document side by side. The distinction it is intended to make visible is not depth or resolution but *edge semantics*: in panels (a) and (b) an edge means *is contained in*, and in panel (c) an edge means *must precede*. Panels (a) and (b) are first-hand screenshots taken on 27 August 2026. They are reproducible only in the sense that the procedure is stated: both products are closed and change without notice, so a reader running the same steps later may see a different rendering. We state the capture date for that reason and make no claim about either product’s behaviour beyond that date.

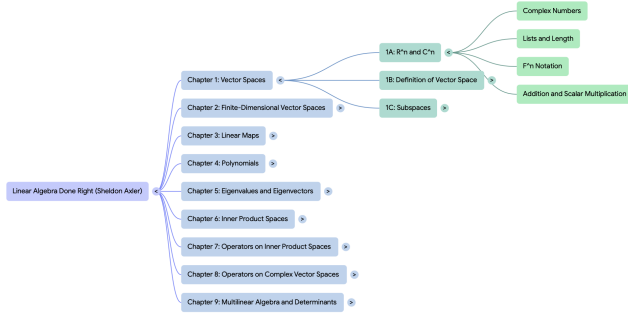
3. METHODOLOGY

3.1. Design

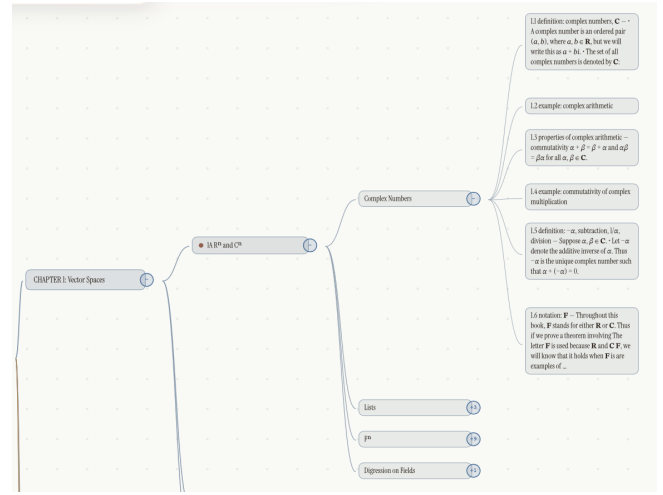
3.1.1. What the base library already provides: Formalising MARD requires first naming what it does not add, because the base library ships a great deal that sounds like our contribution. Its metadata types carry the root model, maximum depth, iteration count, backend and environment; its completion object carries the full execution trajectory—iterations, code blocks, and nested sub-calls, each with its own metadata, recursively; and per-call token and cost accounting is built in.

That metadata, however, flows *upward* and is observational: it is a record of what happened, assembled for inspection after the fact. It never re-enters a prompt and never influences a child call. This is checkable in the source rather than a matter of framing. When a parent spawns a child, the child is constructed with a fresh, empty logger—the library’s own comment says the purpose is that the child’s trajectory be captured, not that the parent’s be communicated—and the child receives only the prompt string that the parent’s REPL code passed to it. Nothing else crosses the boundary.

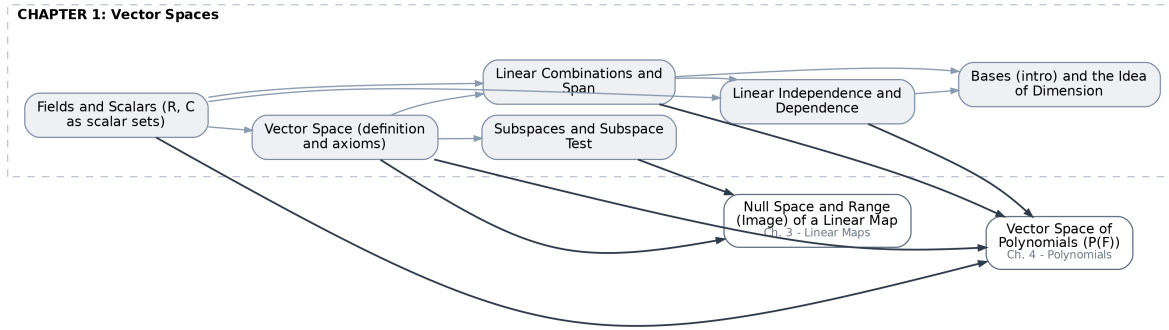
¹Declaration of interest: Tasur is built and maintained by one of the authors. It is included as an illustration of rendering granularity, not as an evaluated system, and no performance claim is made for it in this work. Neither named product is used as a quantitative baseline anywhere in this paper, for the reasons given in Section 2: their architectures are unpublished and they expose no token accounting, so no such comparison would reproduce.



(a) NotebookLM (containment).



(b) Tasur¹ (containment, atomic level).



(c) MARD (precedence).

Fig. 1. Three renderings of *Linear Algebra Done Right*, 4th ed. (abridged text, not the full book), used for the dependency-ordering case study rather than as the primary measurement document. (a) NotebookLM’s mind-map, captured 27 Aug 2026: edges denote containment, leaves are topic labels. (b) Tasur, same day, one branch at full depth (whole-tree leaves are illegible): edges still denote containment but descend to typed atomic items with source text and notation. (c) MARD’s concept graph, same scope and idiom, so only *edge semantics* differ: Chapter 1’s six concepts plus two outside dependents—dark edges cross chapters, grey stay inside. Full graph: 54 concepts, 116 edges, 76 cross-chapter; single run, seed 11. (a)–(b) encode containment already declared by the headings; (c)’s edges assert precedence instead—each a falsifiable prediction, not a restatement of the table of contents.

MARD’s envelope flows *downward* and is operative. Skeleton, accumulated findings and parent directive enter the child’s context before it runs, changing what the child can see and therefore what it produces.

One objection should be anticipated. The base library does expose a slot for injecting a small user-authored string alongside the root’s context, and declines to populate it for children. The rebuttal is about what fills the slot, not whether a slot exists: that parameter is designed for a fixed, short, user-supplied string—its documentation’s own example is the user’s original question. The envelope is accumulated state that grows across calls. No mechanism in the base library constructs, propagates or compounds such state, and none carries one child’s findings to the next. The library gives us somewhere to put an envelope; the envelope is ours.

3.1.2. The metadata envelope: Let D be a document and $S(D)$ a *skeleton*: an ordered structural index of D ’s chapters and sections with their positional spans, derived from D ’s own

text. For a recursive call c we define the envelope

$$E(c) = \langle S(D), F(c), d(c) \rangle, \quad (1)$$

where $F(c)$ is the set of findings accumulated by calls ordered before c , and $d(c)$ is the directive issued by c ’s parent stating the purpose of the call. $E(c)$ is rendered into c ’s prompt in addition to its text slice.

The design intent was that $E(c)$ remain small while $F(c)$ grows monotonically with the number of completed calls: findings are summarised rather than concatenated, and the rendering applies a recency window, so a call sees the most recent findings rather than all of them—which predicts that recovered cross-chapter relations will skew toward nearby chapters. An earlier draft of this manuscript stated a per-call envelope budget of approximately 300 tokens against a ceiling of 500. Measuring the rendered Pass 1 prompts directly refutes that figure, and we report the measurement rather than the intent.

Across the 14 Pass 1 calls of a representative run, the skeleton $S(D)$ renders at 4031 tokens—identical on every

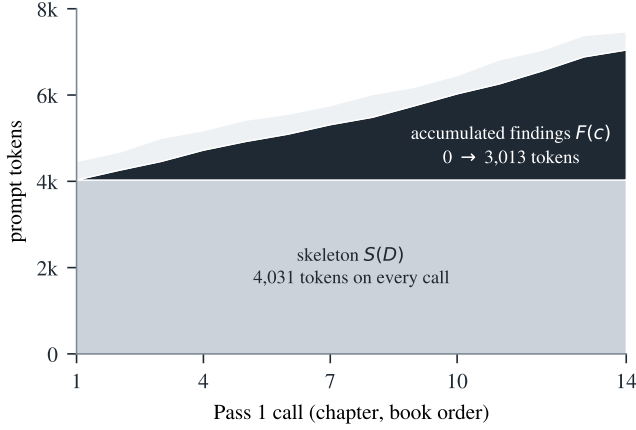


Fig. 2. Composition of the Pass 1 prompt across one run’s fourteen chapter calls, measured from the logged prompts rather than from the design intent. The skeleton is constant at 4031 tokens because it is the same structural index every time; the findings channel grows monotonically from nothing to 3013 tokens as chapters complete. The remaining band is the chapter’s own text slice, the directive and the response schema. The two channels ablated separately in Section 4.5 are the two large bands here, and the larger of them is the one whose removal changes nothing.

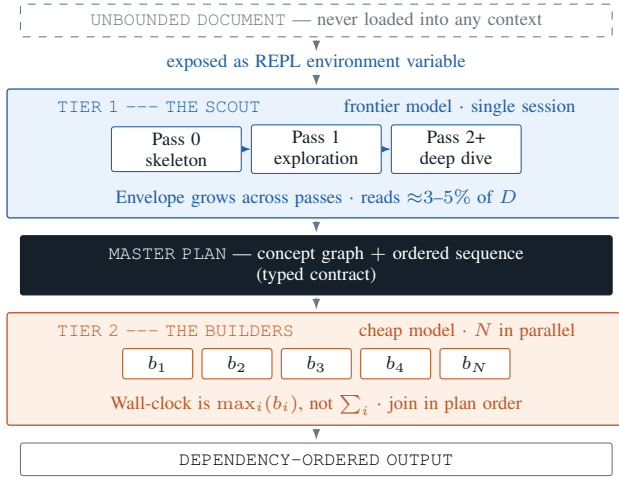


Fig. 3. MARD architecture. The document is never placed in a prompt; it is bound to a REPL variable and explored by the frontier tier, which emits a typed Master Plan that dispatches a parallel budget tier.

call, since it is the same structural index—the accumulated findings $F(c)$ at a mean of 1446 tokens and a maximum of 2853, and the directive $d(c)$ at 56. The envelope is therefore on the order of 5500 tokens per call against a total prompt of 5788, and is not a small annotation beside the text slice but the great majority of what the frontier tier reads. The recency window does bound $F(c)$ as designed; the skeleton, which is not subject to it, does not shrink at all.

This measurement anticipates the ablation result in Section 4.5. The component consuming the most input is the one whose removal costs nothing.

3.1.3. Pass structure: MARD proceeds in passes over D , all executed by the frontier tier.

- **Pass 0 — skeleton extraction.** Derive $S(D)$ from the document’s text. This pass is predominantly arithmetic over detected block structure rather than model inference; a single labelling call assigns topic labels. Crucially, the skeleton is text-derived only: where a source document ships a publisher-supplied bookmark outline, that outline is used solely as a yardstick for evaluating skeleton quality and is never an input (Section 3.2.5).
- **Pass 1 — enriched exploration.** One call per chapter, each carrying $E(c)$ and each contributing to F . The output is the Master Plan.
- **Pass 2 — targeted deep dive.** Revisit spans that Pass 1 marked as uncertain, at greater depth.

This manuscript reports a two-pass configuration. The system evaluated here executes Pass 0 and Pass 1 only; Pass 2 is not run. This is stated here, in every results caption, and in the limitations, because describing a two-pass system as three-pass would be an overclaim. The decision was taken deliberately: Pass 1’s implementation was completed one day before this project’s feature freeze, and the reliability of a pass cannot be evidenced in a day. Layering a targeted deep dive on an unevidenced exploration pass would make a subsequent wrong result undiagnosable—it could not be attributed to either pass. The two-pass configuration is additionally, and by design, the depth-0 point of the depth sweep in Section 4.5, so this manuscript’s number is a valid point on that curve rather than a one-off compromise.

3.1.4. The Master Plan as a typed contract: Pass 1 emits a *Master Plan*: a concept graph together with an ordered study sequence and the rationale for any reordering. It is validated against a schema at the tier boundary, and a malformed plan fails loudly rather than dispatching N subtly incorrect builders.

The field that carries the contribution is the concept graph’s edge set. Each edge asserts that one concept is prerequisite to another, and each is therefore a falsifiable prediction scorable against the document’s own in-text cross-references. Edges are validated at the boundary: an edge naming a concept that no call has declared is rejected and logged rather than silently admitted, which makes the rejection log itself a measurement of what exploration could and could not reach.

3.1.5. Two-tier execution and dependency-ordered joining: The Scout (frontier tier, single session) reads approximately 3–5% of D and emits the Master Plan. The Swarm (budget tier) then runs N builders in parallel, one per section, each receiving its section slice plus its plan directive. Because the builders fork and join, wall-clock time is $\max_i(\text{builder}_i)$ rather than $\sum_i$, and we report both so that the parallelism claim is checkable rather than asserted.

Builder outputs are joined in Master Plan order—the dependency-derived ordering—rather than in book order. We note explicitly that book order is itself an expert baseline, since the document’s author already sequenced the material; reordering must therefore beat a real ordering, not a random one. The join additionally validates non-empty output per plan step and raises rather than returning a partial artefact, so a run

that produced a short result fails visibly instead of scoring as a complete but poor one.

3.1.6. What MARD changes, and what it inherits: Table I sets the two arms side by side. It is grouped deliberately: the upper block is what MARD takes unchanged from the base paper, and only the lower block is ours. Presenting the comparison the other way round—differences first, shared machinery unstated—would let a reader credit MARD with the recursion and the tiering, which are not our contribution (Section 3.1.1).

Three rows in the lower block are worth reading together, because they describe a single property rather than three. *Source binding*, *malformed intermediate* and *empty generation* are the points at which MARD can refuse to proceed. Vanilla RLM has no equivalent of any of them: it has no typed intermediate to validate, and no record of which span a given concept should have been written from, so there is nothing against which a generation step could be checked. That asymmetry is the mechanism behind the faithfulness observation reported in Section 4.4, and it is architectural rather than incidental.

3.1.7. The stated precondition: MARD requires exploitable structure. On a flat context there is no skeleton to extract, F carries no positional meaning, and the envelope reduces to overhead. We therefore predict that MARD degenerates to vanilla-RLM behaviour on flat inputs, and we test this with a negative control rather than omitting it. This is an objective of the work, not an embarrassment to be discovered in review.

The control is a *structure-ablated variant of the primary document*: the same text with its section order shuffled and its heading markers removed. This is a deliberate choice over an external flat-context benchmark. It holds content constant and varies only structure, so it is a manipulation rather than a change of corpus, and it parallels the envelope ablation one level lower—A1 removes the accumulated structure the system builds, and this removes the structure the document supplies. The pipeline treats the resulting empty skeleton as a recorded condition rather than an error, so degeneration appears in the run log as a finding rather than as a missing value. An external flat-context benchmark subset is fixed and frozen in the project’s evaluation artefacts and is run in the full study (Section 5).

3.2. Algorithm and Implementation

3.2.1. The algorithm: Algorithm 1 states MARD end to end. Three lines carry properties argued for elsewhere in this section and are worth reading as such. Line 3 is the boundary condition of Section 3.1.7: a document from which no skeleton can be derived returns a recorded degenerate condition rather than an error, so graceful degeneration appears in the run log as a finding. Line 9 is the validation boundary—an edge naming a concept no call has declared is rejected and logged, which makes the rejection log itself a measurement of what exploration could not reach. Line 16 is the source binding: a builder’s brief is constructed from the plan’s recorded span,

Algorithm 1 MARD (two-pass configuration)

Require: document D with section index; frontier model M_1 ; budget model M_2

Ensure: dependency-ordered artefact, or a loud failure

```

1:  $K \leftarrow \text{SKELETON}(D)$  {Pass 0; text-derived only}
2: if  $K = \emptyset$  then
3:   return DEGENERATE {O4 boundary; recorded, not an error}
4: end if
5:  $E \leftarrow \text{ENVELOPE}(K, F=\emptyset, d=\text{nil})$ 
6: for each chapter  $c \in K$  in book order do
7:    $E_c \leftarrow E.\text{FORCHILD}(c, \text{DIRECTIVE}(c))$ 
8:    $R \leftarrow M_1(\text{RENDER}(E_c) \parallel \text{TEXT}(c))$  {Pass 1}
9:    $C_c, X_c \leftarrow \text{VALIDATE}(R, \text{KNOWN})$  {reject undeclared}
10:   $E \leftarrow E.\text{WITHFINDINGS}(c, C_c, X_c)$  {envelope grows}
11: end for
12:  $P \leftarrow \text{COMPILEPLAN}(C, X, K)$  {typed Master Plan}
13: if  $P$  fails schema validation then
14:   fail loudly {never dispatch on a malformed plan}
15: end if
16:  $B \leftarrow \text{BRIEFSFOR}(P)$  {each brief carries  $P$ ’s recorded span}
17: for all  $b \in B$  in parallel do
18:    $s_b \leftarrow M_2(\text{PROMPT}(b))$  {Tier 2; wall-clock  $\max_i$ }
19: end for
20: if any  $s_b$  is empty then
21:   raise INCOMPLETEARTEFACT
22: end if
23: return JOIN( $\{s_b\}$ ,  $P.\text{SEQUENCE}$ ) {plan order, not book order}

```

so generation never depends on a model matching text to a concept.

3.2.2. Model pair, parity and cost: The frontier tier (Scout) is gpt-5.2; the budget tier (Swarm, and the baseline’s sub-calls) is gpt-5-mini. Rates were read from the provider’s own pricing page on the day the rate card was written and are recorded once, in code, with their source and retrieval date; the accounting layer refuses to price a run against a rate more than thirty days old rather than let a stale figure ride.

We are explicit that this pair is selected on *role fit and cost*, not on a published benchmark comparison. The Scout runs once per document and must hold the skeleton and accumulated envelope across a sequence of chapter calls; the Swarm is N independent builders each consuming one section and a directive, where call volume rather than per-call ceiling dominates. The budget tier carries roughly 93% of the token volume at approximately one seventh the rate on both input and output. A model sweep that would test that reasoning is deferred to the full study, and we do not present the selection as a measured result.

Both arms are executed on a single implementation. Isolating the envelope requires that the two arms differ *only* in its presence. Executing the baseline on a separate code-base would confound the envelope with two independent

TABLE I
ARCHITECTURAL COMPARISON OF THE TWO ARMS. BOTH RUN ON ONE IMPLEMENTATION, AT MATCHED RECURSION DEPTH, ON IDENTICAL INPUTS.
THE UPPER BLOCK IS INHERITED FROM THE BASE PAPER AND IS NOT CLAIMED AS A CONTRIBUTION

Aspect	Vanilla RLM (B1)	MARD
Inherited from the base paper — identical in both arms, and not our contribution		
Document placement	Bound to a REPL variable; never enters a prompt	Identical
Model tiering	Frontier root, budget sub-calls	Frontier Scout, budget Swarm — the same economics
Recursion depth	Sub-calls are plain completions	Matched, by construction
Input	Cleaned text stream; front matter excluded, publisher outline withheld	Identical, by construction
Where MARD differs		
Exploration strategy	Authored at run time by the root model as REPL code	Fixed pass structure; the model supplies findings, not control flow
Structural map	None. Any structure must be re-derived by code the root writes	Pass 0 skeleton, derived from the document’s own text
What a sub-call receives	A raw text slice	Slice <i>plus</i> skeleton, accumulated findings, and a parent directive
State across calls	None; each sub-call is independent	Envelope accumulates; a later chapter sees earlier chapters’ findings
Direction of metadata	Upward, observational — recorded after the fact	Downward, operative — enters the prompt before the call runs
Unit of dispatch	Chosen by the root at run time	Chapter for exploration; section for generation
Intermediate artefact	None; free text inside the root’s own history	Typed Master Plan: concept graph, ordered sequence, reordering rationale
Source binding at generation	The model matches text to a concept at generation time	The brief is constructed from the plan’s recorded source span
Output ordering	Whatever the root emits	Master Plan order, i.e. dependency order
Malformed intermediate	No intermediate to validate; execution proceeds	Schema validation at the tier boundary; a malformed plan fails loudly
Empty generation	Included in the output; run reports success	Join validates non-empty text per plan step and raises
Parallelism	As the root’s code chooses	N builders fork-join; wall-clock is $\max_i$, not $\sum_i$

implementations—differing prompt templates, REPL surfaces and stopping criteria—none of which is the variable under study. We therefore implement vanilla RLM as an envelope-disabled configuration of the same system, and treat the reference implementation accompanying the base paper [1] as the specification we implement against rather than as the artefact we measure.

The cost of this choice is that our baseline is our own implementation of RLM rather than the original authors’. We mitigate it in the only way available: by reproducing a published result from the base paper first-hand on our own harness before any number in Section 4 is trusted. That check is not a formality here—it is the sole evidence that the vanilla arm faithfully reproduces the published method. **This check was not performed.** The provider migration described in Section 3.2 consumed the time allocated to it, and we report its absence rather than substituting a weaker check and calling it done. The consequence is stated plainly: we have no first-hand evidence that our vanilla arm reproduces the published method’s performance, only that it implements the published method’s mechanism, which our tests assert directly. Every comparison in Section 4 should be read as against *our implementation of RLM*. This is the single largest threat to the baseline’s validity in this manuscript and it is unresolved.

What we did run is a bounded harness check, and we are careful about what it does and does not establish. Six tasks from the frozen OOLONG subset were executed end to end through the vanilla arm and scored with the base paper’s own scoring function, imported unmodified from the reference implementation rather than reimplemented. All six executed, and all six produced output the reference scorer could parse. **We report no accuracy estimate from six tasks and draw no comparison to the published figure**—at that sample size the confidence interval spans both the published RLM result and the base model’s, so a number would carry the appearance of evidence without its substance. What the check rules out is narrow and worth stating anyway: the failure mode in which our harness cannot execute the published benchmark at all, or emits output the reference metric cannot read. It leaves the paragraph above unchanged.

Concurrency and sub-call size are bounded, identically for both arms. Runs were executed under a provider rate-limit tier that required bounding sub-call concurrency and sub-call input size. Sub-call concurrency was bounded at 4 simultaneous calls, and Tier 2 generations at 4,096 completion tokens with reasoning effort set to `low`—the latter after 15 of 84 builders in an early run spent an entire 2,048-token budget on reasoning and returned empty artefacts, which the typed

TABLE II
COST OF THE EVALUATION MATRIX ON THE PRIMARY DOCUMENT. CORPUS QUANTITIES AND THE BASELINE’S PER-RUN COST ARE MEASURED; THE MARD FIGURE IS PROJECTED FROM THOSE QUANTITIES AT THE SAME RATES

Measured quantity	Value
Primary document, characters	2 468 431
Primary document, tokens [†]	≈617 000
Sections (Swarm builders)	120
Chapters explored by Pass 1	14
Published rates, per 1,000,000 tokens	
Frontier tier, input / output	\$1.75 / \$14.00
Budget tier, input / output	\$0.25 / \$2.00
Cost per run	
Vanilla RLM, measured [‡]	\$0.32–\$1.60
MARD, projected	≈\$0.94
Remaining matrix, projected	≈\$11

[†]Derived from character counts at an assumed ratio; pending confirmation with a tokeniser. [‡]Three logged runs; the spread is reported rather than averaged, for the reason given in Section 4.2.

TABLE III
CORPUS DOCUMENTS AND LICENCES, VERIFIED AGAINST EACH SOURCE’S OWN COPYRIGHT PAGE

Document	Licence	Role
Introduction to Computer Science (OpenStax)	CC BY 4.0	Primary
University Physics, Vol. 1 (OpenStax)	CC BY-NC-SA 4.0	Secondary
Linear Algebra Done Right, 4th ed.	CC BY-NC 4.0	Secondary

join surfaced as an error rather than as a short document. Both bounds are applied to *both* arms and are recorded in every run’s configuration snapshot. We report this because it is not a neutral implementation detail: throttling can degrade output completeness, and applying it asymmetrically would manufacture a difference between the arms that has nothing to do with the envelope.

The cost model is instantiated with measured token counts rather than a claimed multiple. Table II reports the projected cost of the full evaluation matrix from measured corpus quantities. We report the model rather than a headline ratio because provider rates move month to month, and a fixed multiple quoted against a rate that later disappears is precisely the kind of claim that decays into misinformation.

3.2.3. Corpus and licensing: Our corpus consists of openly licensed textbooks spanning distinct domains, under an open-license-only constraint on all source material. Table III summarises them. Licensing was verified against each publisher’s own copyright page rather than a third-party aggregator.

OpenStax titles were selected specifically because they ship machine-readable learning objectives, glossaries and answer keys—the substrate required for document-native ground truth extracted without human annotation, and therefore without an

TABLE IV
PARSE QUALITY, PRODUCED BY THE INGESTION PIPELINE AND REPRODUCIBLE FROM THE CORPUS ARTEFACTS. ORPHAN PUNCTUATION COUNTS PUNCTUATION STRANDED WHERE A FORMULA FAILED TO EXTRACT

Measure	intros	physics1	axler
Pages	939	959	404
Characters after cleaning	2 432 812	2 032 210	737 844
Headings detected	1 403	1 081	17
Learning-objective blocks	61	99	0
Key-terms blocks	14	17	0
Orphan punctuation (per 1k lines)	30.3	113.5	87.8
Warnings raised	0	1	3

ethics-approval process this project’s timeline cannot absorb. **On the licence of the primary document.** *Introduction to Computer Science* is CC BY 4.0 (plain attribution), confirmed against OpenStax’s book-level licensing page. Individual chapter footers within the same book state CC BY-NC-SA; we treat the book-level page as authoritative and record the inconsistency rather than silently choosing one.

3.2.4. Ingestion and primary-document selection: Primary-document selection was made on measured evidence rather than convenience. Table IV reports parse quality across the three candidates. No optical character recognition is used on any document: all three carry a text layer, and OCR would replace exact text with approximate text—decisively, OCR errors in glossary terms would corrupt the very reference set against which task quality is scored, silently and irreversibly.

University Physics is disqualified as primary on content loss, not formatting loss: its orphan-punctuation rate is nearly four times that of the primary document, and inspection confirms sentences from which equations have vanished entirely. That damage is not uniform across systems under test—a configuration that reads more of the document encounters more of it—and would manufacture a difference between MARD and vanilla RLM having nothing to do with either. *Linear Algebra Done Right* carries no learning objectives and no key-terms sections, so document-native ground truth cannot be extracted from it; it remains the strongest candidate for the dependency-ordering case study, which is scored separately.

Page mapping was verified by sampling 1200 blocks at random across the three documents and confirming each appears on the page it claims; 1199 passed, the single failure being a whitespace-normalisation difference in an index line rather than a mapping error.

3.2.5. Two structural confounds, and how they are controlled: Both concern structure reaching a model without that model having derived it. If structure arrives by another route, we would be reporting the publisher’s work as MARD’s.

- 1) The publisher’s bookmark outline.** All three documents ship one. The ingestion pipeline writes it to a separate artefact with an explicit provenance field and never merges it into the text stream, so that using it

must be a deliberate decision rather than an accident. It serves as the yardstick for skeleton quality and is not an input to Pass 0.

- 2) **The printed table of contents inside the body text.** Subtler, because it arrives through the ordinary text channel: the primary document prints its full chapter list on page 7, so a model reading the opening pages obtains the skeleton for free. Front matter is detected, tagged, and excluded from the text stream supplied to every system.

Both controls are applied identically to the vanilla-RLM arm. If the two arms received different inputs, the comparison would not isolate the envelope.

3.2.6. Skeleton fidelity: Because the skeleton is the substrate for everything downstream, its quality is measured before it is used. Against the publisher outline, the primary document’s text-derived skeleton achieves 82.2 % recall at 0.0 pages of boundary error. Boundary error of zero indicates that where a section is recovered, its span is recovered exactly; the recall shortfall is missed sections, not misplaced ones.

3.2.7. Systems compared:

- **MARD** — two-pass, as specified above.
- **B1, vanilla RLM** — the isolation baseline: the base paper’s architecture, a root REPL loop issuing flat sub-calls, on identical models, document and structural controls.
- **A1, envelope removed** — MARD’s architecture with the envelope emptied. This is a *separate* run from B1 and serves a different purpose, discussed below.
- **Negative control** — MARD and B1 on the structure-ablated variant of the primary document, testing the graceful-degeneration half of the claim under content held constant.

B1 and A1 are deliberately distinguished, because they answer different questions. B1 compares MARD’s architecture against the base paper’s, and a win there changes several things at once—the envelope, the pass structure, the typed plan, two-tier dispatch and dependency-ordered joining. A1 holds the architecture fixed and removes only the envelope, and is therefore the only comparison that attributes an effect to the envelope specifically, which is what this paper’s claim rests on. Reporting B1 alone would license the weaker conclusion that our pipeline is better, not that the envelope is why.

The full study additionally runs full-context, naive chunking and embedding-RAG baselines first-hand.

Recursion depth is held fixed. Both arms run at a matched recursion depth in which sub-calls are plain completions. This is the base paper’s primary reported condition. Holding depth fixed is not incidental: if the arms differed in depth, the measured difference would confound the envelope with an extra layer of recursion, and the isolation claim would be void. We resolve the correspondence architecturally, and disclose that the run logs do not independently confirm it. The control arm runs at `max_depth = 1`: a root call plus one level of delegated sub-calls. MARD’s Tier 1 Scout issues one Pass 0 labelling call and one exploration call per chapter,

TABLE V
RECORDED PER RUN. A NUMBER NOT TRACEABLE TO ALL OF THESE
FROM A LOGGED RUN IS NOT ADMITTED

Field	Definition
Task score	Explanation quality against document-native ground truth
Tokens	Input and output, summed over every model call, logged separately
Calls issued	Split by tier
Cost	At the same-day published provider rate
Wall-clock	Both $\max_i$ and $\sum_i$ over builders, so the parallelism claim is checkable
Run identifier	System, document, repeat
Config snapshot	Model identifiers, prompt-template versions, depth, active ablation, document, library revision
Additionally, for systems producing per-concept output	
Groundedness rate	Grounded concepts as a fraction of the total (Section 4.4)
Regenerations	Concepts with ≥ 1 discarded prior attempt

and dispatches Tier 2 builders; this is likewise one level of delegation, fixed by the architecture rather than configurable, which is why MARD’s configuration snapshot records no depth parameter. The two arms therefore match on the property that matters—neither nests delegation more than one level deep.

The disclosure is that MARD’s call log records every call at depth 0, including all 84 Tier 2 builders, and `parent_call_id` is unpopulated in both arms. The control arm does distinguish its root from its sub-call. A reader cannot therefore verify the depth match from the logs alone, only from the call roles and the source; the correspondence above rests on the latter. This does not satisfy our own measurement protocol, which requires depth to be recorded as an (`enable_sub_calls`, `max_recursion_depth`) pair precisely so that a matched-depth claim is checkable rather than asserted. We record it as an instrumentation defect found during manuscript preparation and not repaired before the results freeze.

3.2.8. Metrics: Every run of every system records the fields in Table V. A number that cannot be traced back to all of them from a logged run is not admitted to this paper.

Task score. The evaluated output modality is explanations only. Flashcards, quizzes and diagrams are demonstrable prototype features that test nothing about the envelope and would multiply the evaluation surface; they are excluded from measurement. Explanations are scored against document-native ground truth extracted programmatically. Of the ground-truth sources available in the corpus, this manuscript uses *per-chapter learning objectives* and *in-text cross-references*. Glossary terms are present in the corpus but are typeset as an undelimited run-on list, and recovering term boundaries reliably requires font-level information we defer to the full study; they are therefore *not* used here. We state this because a restriction disclosed is a scope decision and a

restriction implied is not. Matching is stemmed token-set overlap, scored as recall of the objective’s content tokens against the candidate explanation, with stopwords and tokens under three characters removed. The reported threshold is 0.6; every figure is additionally computed at 0.5 and 0.7 and the ranking of configurations is unchanged at all three. Raw matched and reference token counts are retained per objective in the results log, so a reviewer who rejects the threshold can recompute from the same artefacts without an API key. Scores are computed per chapter and objective-weighted, not over the whole document: at whole-document granularity every configuration scores between 0.94 and 0.97 and the metric does not discriminate.

Dependency-ordering score. Scored independently of task quality, as forward-reference violations: the number of times a generated sequence places a concept before something the document itself declares as its prerequisite, counted for book order and for Master Plan order.

Variance. Every reported number is the result of three independent repeated runs, with spread reported alongside the central value. We report repeats rather than fixed random seeds deliberately: the inference endpoints used here do not expose deterministic decoding, so a nominal seed would name a property the system does not have. Where variance swamps an effect, that is reported as the finding.

4. RESULTS AND DISCUSSION

4.1. Isolating the envelope

Table VI reports the four configurations on the primary document, three repeated runs each, at matched recursion depth. Task score is objective-weighted per-chapter coverage of the document’s own learning objectives at a stemmed-overlap recall threshold of 0.6 (Section 3); the two neighbouring thresholds are reported in the supplementary results log, and the ranking is stable across all three.

The frozen ablation set specified A1 as “the envelope removed.” We report it as two separable manipulations, because the envelope carries two distinct channels: the *skeleton*, a structural map of the document, and the *accumulated findings*, the record of what earlier chapters established. A1s suppresses the skeleton; A1f suppresses the findings. Reporting them separately is what makes the result in Section 4.5 legible.

MARD’s mean task score exceeds the baseline’s by 0.070, and its mean input token count is 15.8× lower. Neither figure supports a claim that MARD produces better explanations. The baseline’s best repeat scores 0.765, above MARD’s best of 0.650; with three repeats and ranges that overlap this far, the difference in means is not a difference we can defend. We state the task-score comparison as **indistinguishable**, and locate the contribution elsewhere.

The token reduction requires a similar qualification. MARD’s Tier 2 builders receive a concept label, a section identifier and a page range, and never the document’s prose (Section 3.1). Part of the 15.8× reduction is therefore attributable to MARD not reading the text at the point of generation, rather than to reading it more efficiently. We report

TABLE VI
THE FOUR CONFIGURATIONS ON THE PRIMARY DOCUMENT. THREE REPEATED RUNS EACH; MEAN WITH MIN-MAX IN PARENTHESES. NO REPEAT WAS EXCLUDED

System	Task score	Input tok.	Cost
B1, vanilla RLM	0.551 (0.315–0.765)	1,493,033 (517k–3,315k)	\$0.832 (0.32–1.60)
MARD (two-pass)	0.621 (0.584–0.650)	94,242 (93.8k–94.7k)	\$0.578 (0.57–0.59)
A1s, skeleton sup-pressed	0.730 (0.720–0.745)	37,631 (36.4k–38.6k)	\$0.498 (0.49–0.50)
A1f, findings sup-pressed	0.623 (0.560–0.658)	78,064 (77.5k–78.6k)	\$0.524 (0.51–0.53)

TABLE VII
STRUCTURAL STABILITY AND PROVENANCE, THREE REPEATS EACH. CONCEPT COUNTS ARE POST-MERGE; DUPLICATE DECLARATIONS ARE MERGED RATHER THAN REJECTED. CITATIONS ARE SOURCE REFERENCES THAT RESOLVE AGAINST THE CORPUS

System	Concepts / sections	Ch. cov.	Cites
B1, vanilla RLM	156, 190, 75	14, 14, 9	0, 0, 0
MARD (two-pass)	84, 83, 84	14, 14, 14	84, 83, 84

the reduction as a property of the architecture as specified, not as evidence of superior retrieval, and return to it in Section 4.9.

4.2. Variance across repeats

The spread in Table VI is the result. Under identical configuration on an identical document, the baseline’s task score ranges from 0.315 to 0.765—a spread of 0.450, or 82% of its own mean. MARD’s ranges from 0.584 to 0.650, a spread of 0.066. The baseline’s input token count varies by a factor of 6.4 across the same three repeats (517,000 to 3,315,000); MARD’s varies by 0.9%. Its cost varies 5.0×; MARD’s by 3.8%.

This instability is visible in the artefacts themselves. Table VII reports the organisational structure each system imposed. The baseline emitted 156, 190 and 75 concept-level sections on its three repeats, and its chapter-level organisation shifted from 20 headings, to none at all, to a single one. MARD emitted 84, 83 and 84 concepts, each carrying a verified section identifier and page range.

Two properties follow, and we are careful to distinguish them. MARD’s *scaffold* is near-deterministic: the concept set is computed from a skeleton derived from the verified corpus rather than sampled, and varies only where a duplicate-identifier merge fires. Its *elaboration* is not: edge counts range 124–133, and never-declared-prerequisite rejections range 0–4. The claim is a deterministic scaffold with variable elaboration, not a deterministic system.

Provenance separates the two arms without qualification. Every concept MARD emits carries a source section and page range, and none of the 677 Tier 2 calls across the nine MARD-family runs cited a section it was not assigned. The baseline emitted no resolvable source reference in any repeat. We note

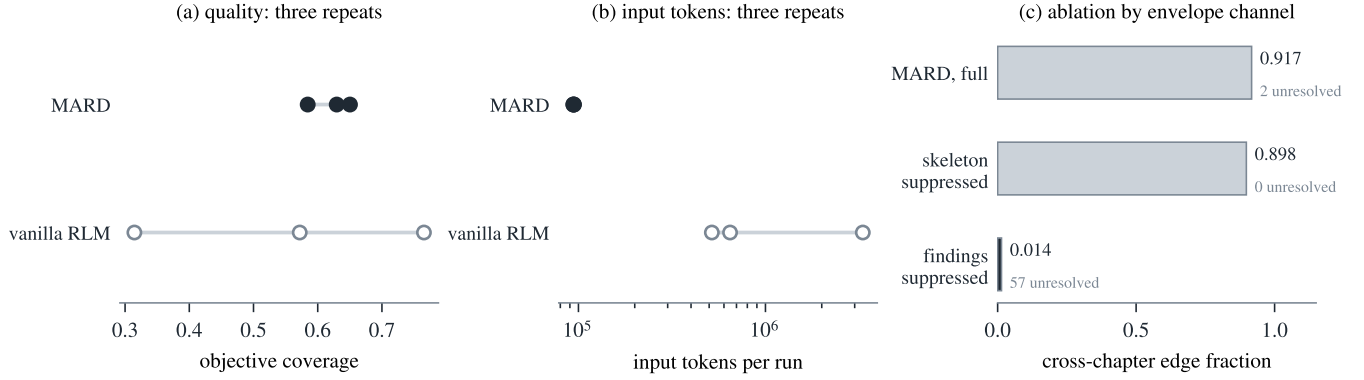


Fig. 4. Three repeated runs per configuration on the primary document, each point one run. (a) Objective coverage: the baseline’s three repeats span 0.315–0.765 while MARD’s span 0.584–0.650; the distributions overlap, which is why the quality comparison is reported as a null. (b) Input tokens, log scale: MARD’s three points coincide at this resolution because they differ by under 1%, while the baseline’s span a factor of 6.4. (c) Ablation by envelope channel, with the count of prerequisite declarations naming a concept that was never declared. Suppressing the skeleton changes neither quantity; suppressing the accumulated findings collapses cross-chapter linkage and leaves a mean of 57 unresolved declarations per run.

that MARD’s citations are verified as *attribution-correct*—the pointer matches the assignment—and not as text-grounded: no builder call carried the cited prose, so a citation certifies where a claim was supposed to come from, not that the generated text was derived from it. Section 4.4 develops this distinction, which we regard as the study’s most consequential finding.

No repeat was excluded from any figure above. Four earlier MARD runs on seed 11 predate the asynchronous-seam and token-budget corrections described in Section 3.2 and are not results; the run identifiers of every included and excluded run are listed in the supplementary results log.

4.3. Replication on a second document

On a second document MARD’s advantage widens (Table VIII): it consumes 20.6× fewer input tokens than the baseline, against 15.8× on the primary document, while emitting a concept set that varies by one across three repeats where the baseline emits a different document format on each.

We repeated both arms, three seeds each, on *Linear Algebra Done Right* (404 pages, 10 numbered chapters). That document carries no learning-objective blocks, so no task score exists for either arm on it; what it tests is whether the reliability, cost and structural findings hold on a second corpus. Two of the three replicate and one does not, and we report the failure alongside the successes.

Structural stability replicates, and more starkly. MARD emitted 54, 54 and 53 concepts. The baseline emitted three differently *formatted* artefacts: 86 flat numbered items on one seed, 9 nested numbered items on the second, and 10 second-level plus 67 third-level markdown headings on the third, at 9417, 13,524 and 4032 words respectively—a 3.35× swing in length and a different document format on each run. On this document the instability is visible without any metric at all.

The token reduction replicates and widens. Mean input tokens were 448,915 for the baseline against 21,761 for MARD, a factor of 20.6×, against 15.8× on the primary document.

TABLE VIII
THE PRIMARY DOCUMENT’S FINDINGS RE-MEASURED ON A SECOND DOCUMENT. THREE REPEATS EACH. *Linear Algebra Done Right* CARRIES NO LEARNING-OBJECTIVE BLOCKS, SO NO TASK SCORE EXISTS FOR EITHER ARM ON IT

	intros 939 pp., 14 ch.	axler 404 pp., 10 ch.
Input tokens, mean		
B1, vanilla	1,493,033	448,915
MARD	94,242	21,761
reduction	15.8×	20.6×
Structural units, per repeat		
B1, vanilla	156, 190, 75	86, 9, 77
MARD	84, 83, 84	54, 54, 53
Spread across repeats		
B1 input tokens	6.4×	1.05×
MARD input tokens	1.01×	1.07×
B1 output words	1.5×	3.35×
MARD cross-chapter fraction		
range	0.895–0.930	0.655–0.673

The token-variance finding does not replicate. On the primary document the baseline’s input token count varied by a factor of 6.4 across three identical repeats. On this document it varied by 1.05—essentially not at all—while MARD’s varied by 1.07. The baseline’s cost still varied 1.26× and its wall-clock 2.43×, but the dramatic input-token instability of Section 4.2 is specific to the primary document and we withdraw any suggestion that it is a general property of the method. The likely mechanism is that the primary document is roughly eight times larger, leaving the root model far more scope for its exploration to diverge; we did not test that explanation and offer it as a conjecture, not a finding.

Cross-chapter linkage is markedly lower. MARD’s cross-

TABLE IX
GROUNDEDNESS BY SYSTEM AND REPEAT. “ROOT-AUTHORED” MEANS THE ROOT MODEL WROTE THE CONCEPT ITSELF WITH NO DELEGATED SUB-CALL TO RESOLVE. A RATE IS REPORTED ONLY WHERE DELEGATED CALLS EXIST TO CLASSIFY

System	Seed	Concepts	Grounded	Regen.
B1, vanilla	11	156	— (all root-authored)	0
B1, vanilla	23	190	— (all root-authored)	0
B1, vanilla	42	75	34 / 75 = 0.453	37
MARD	11	84	0 / 84	0
MARD	23	83	0 / 83	0
MARD	42	84	0 / 84	0

chapter edge fraction was 0.655–0.673 on this document against 0.895–0.930 on the primary one. The ablation’s *direction* is not tested here—A1s and A1f were not re-run on this corpus—so we make no claim that the envelope’s mechanism transfers unchanged. What the numbers establish is that the fraction is document-dependent, and that the 0.917 figure in Section 4.5 should be read as a property of one book rather than a constant of the method.

4.4. Groundedness of generated content

The most consequential observation in this study was not a score. In one repeat of the baseline, the system completed without error, reported success, and produced a coherent study guide in which a substantial fraction of the explanations had been generated *without the document in context at all*.

The mechanism is worth stating precisely, because it is a property of the method rather than of our harness. In vanilla RLM the root model authors its own exploration code. In this run that code mismapped a concept onto an unrelated passage; the resulting generation failed to parse and was discarded; and the retry proceeded with an empty source field, producing an explanation from the model’s parametric knowledge. Nothing in the returned artefact, the exit status, or the aggregate token accounting distinguishes that explanation from a grounded one. The failure is visible only by walking the execution trajectory.

We therefore instrument groundedness directly. For each concept in a final artefact we resolve the generating call and classify it as *grounded* (non-empty source text), *ungrounded* (empty or absent source), or *regenerated* (at least one prior attempt discarded). This instrumentation was written *after* the behaviour was found by manual trajectory inspection, and we report it as such.

Table IX reports the detector’s output, and it requires two disclosures that a rate column alone would obscure.

First, the baseline’s recursion did not occur on two of its three repeats. On seeds 11 and 23 every concept in the final artefact was authored by the root model directly, with no delegated sub-call to resolve against the document, so there is no generating call to classify and no rate to report. Only on

seed 42 did delegation occur, and there the grounded fraction is 0.453: 34 of 75 concepts were generated with document text in context, 41 were not, and 37 required at least one discarded attempt. A method whose central mechanism is recursive delegation executed that delegation in one run out of three, and reported success in all three. This is the same instability Section 4.2 measures, observed at the level of control flow rather than of output.

Second, and against our own prior expectation: **MARD’s grounded count is zero in every repeat**. No Tier 2 builder call in any of the nine MARD-family runs carried the document’s prose. Inspection of every full, untruncated builder prompt confirms this is by construction—`orchestrate.lm_builder.prompt_for` sends a concept label, a section identifier, a page range, a sequence position and a directive, and never an excerpt. Every MARD explanation is therefore generated from parametric knowledge, prompted by a citation.

We had predicted a groundedness rate near 1.0 for MARD. That prediction conflated two distinct properties, and the distinction is the finding:

- *Text-grounding* — was the cited prose in the generating context? For MARD: never, 0 of 677 builder calls.
- *Attribution correctness* — does the citation identify the span the concept was assigned from? For MARD: always, 677 of 677, verified against the corpus, with 0 mis-cited and 0 regenerations.

MARD therefore offers a weaker guarantee than we claimed and a different one than the baseline offers. It cannot assert that its text derives from the document. It can assert, checkably and for every sentence, where that text was supposed to come from, so a reader or reviewer can go to the named page and disagree. The baseline offers neither: it emitted no resolvable source reference in any repeat, and on the one repeat where delegation happened, 41 of 75 explanations came from an empty source field while the exit status reported success.

The honest characterisation is that MARD converts silent ungroundedness into declared, auditable ungroundedness. That is a real property and it is the one the architecture actually delivers—but it is auditability, not fidelity, and we correct our earlier framing accordingly. Supplying the cited span to each builder is a change of a few lines and would make text-grounding measurable rather than structurally absent; Section 5 records it as the first item of future work, and we note that we did not make it in time to report it here.

MARD’s remaining structural contributions to faithfulness stand independently of grounding: builder inputs are constructed from the plan’s recorded source span rather than matched at generation time; the plan is schema-validated at the tier boundary, so a malformed plan fails loudly rather than dispatching many subtly incorrect builders; and the join validates non-empty output per plan step, raising rather than returning an artefact that is short by a section while satisfying every identity check.

4.5. Ablation

The frozen ablation set comprises four manipulations: A1, the envelope removed; A2, the plan withheld from the Swarm, in which the Master Plan is still computed and still determines join order but builders receive no directive; A3, reordering disabled, joining in book order; and A4, a sweep over recursion depth reported as a curve. This manuscript reports A1, split into its two channels as described in Section 4. A2, A3 and A4 were not run for this manuscript and appear in the full study; we report that as a gap rather than inferring their outcome.

Table X gives the ablation on the mechanism the envelope is built to produce: cross-chapter dependency edges, expressed as a fraction of all accepted edges, together with the count of prerequisite declarations rejected because the named prerequisite was never declared anywhere in the plan.

The two channels behave entirely differently, and this is the manuscript’s central mechanistic result.

Suppressing the accumulated findings collapses cross-chapter linkage from 0.917 to 0.014, and never-declared-prerequisite rejections rise from a mean of 2 to a mean of 57. With no record of what earlier chapters established, the exploring model names prerequisites that exist nowhere in the plan, and the compiler rejects them. The downward-flowing findings channel is therefore not an optimisation over the base library’s upward, observational metadata; it is the component that makes cross-chapter structure possible at all. This is the claim of Section 1, and it is the one result in this study that is large, tightly bounded, and unambiguous.

Suppressing the skeleton does not degrade the system. Cross-chapter linkage falls from 0.917 to 0.898, a difference smaller than either configuration’s own spread across repeats. Task score *rises*, from 0.621 to 0.730, and is the highest of any configuration measured, with the tightest spread. Never-declared rejections fall to zero in all three repeats. Cost falls from \$0.578 to \$0.498, and the frontier tier’s own input consumption falls from 84,811 tokens to 28,147—a reduction of 66.8%, against 60% of the run’s total input. The skeleton is the single most expensive item in MARD’s prompt budget, and removing it improves every quantity in Table X that we can measure.

We report this without adjustment. On every measure available to us the skeleton channel either fails to contribute or actively costs, while consuming the majority of the expensive tier’s input. The honest reading is that MARD’s benefit is carried by one of its two mechanisms, and that the architecture as specified is larger than the architecture that works. We did not anticipate this, and we note that it was the ablation design—not the headline comparison—that surfaced it. Two qualifications bound the finding: the skeleton’s contribution may be to *robustness* on documents whose structure is less regular than the primary document’s, which a single-document study cannot test; and A1s was measured on three repeats of one corpus, so the task-score advantage in particular warrants replication before it is treated as settled. Section 5 states the redesign this implies.

4.6. Negative control: structure ablated

The negative control holds content constant and varies only structure: the primary document’s sections are shuffled and heading markers stripped, leaving the same prose in a scrambled order. Page markers are retained but renumbered sequentially in the shuffled order, so the baseline’s chunking mechanism still functions while carrying no information about the original sequence. The prediction registered in advance was a *null difference*: with no structure to exploit, MARD should reduce to the baseline.

The prediction was wrong, and the manner in which it was wrong is the most informative result in this study.

MARD does not degrade on the ablated document; it declines to run. Pass 0’s skeleton extraction recovers no sections, the envelope renders empty, and the run terminates with `compiled: false` before any model call is issued. All three repeats are identical: zero concepts, zero edges, zero tokens, zero cost, and a wall-clock time under 0.1 seconds. There is no stochastic component left to vary, because nothing stochastic executes.

The baseline, given the same scrambled text, produced a complete and confident study guide on all three repeats. We read the full text of each answer rather than sampling it. None of the three acknowledges disorder, incompleteness, or uncertainty about the input at any point. Two of the three open with the heading “1.1 Computer Science”—the intact book’s actual first section—despite receiving input in which no such ordering, numbering or heading survives. On the third repeat the baseline emitted 47,810 words, nearly three times its output on the intact document, and consumed 1,056,353 input tokens.

The baseline is therefore reconstructing a plausible textbook from its own parametric priors and presenting the reconstruction as a reading of the supplied document. Nothing in its output distinguishes this from the case where the document genuinely had that structure. This is not a scoring difference; it is a difference in what the two systems will assert. MARD’s typed, validated plan makes an unfounded artefact unconstructible—the pipeline fails at the skeleton rather than proceeding without one—while the baseline has no representation in which the absence of structure could be detected, and so cannot report it.

We state two limits on this claim. First, the comparison is one of *architecture*, not of prompt engineering: a baseline instructed to report disorder might well do so, and we did not test that condition. Second, flattening raises the chunk count from 916 to 1,008 and lowers mean chunk size by 9%, so the manipulation varies chunking granularity slightly as well as structure; the effect is far too small to account for an outcome this categorical, but it is a deviation from a pure single-variable manipulation and we record it as one.

4.7. The structure-dependence boundary

Taken with Section 4.5, the boundary is sharp in both directions. Where structure is present and the findings channel propagates, MARD produces a dependency-ordered artefact with 91.7% cross-chapter linkage. Where the findings channel

TABLE X

ABLATION A1, BY ENVELOPE CHANNEL. CROSS-CHAPTER FRACTION IS CROSS-CHAPTER EDGES OVER ALL ACCEPTED EDGES. NEVER-DECLARED REJECTIONS COUNT PREREQUISITES NAMED BUT NEVER DECLARED. TIER 1 INPUT IS THE FRONTIER TIER’S OWN TOKEN CONSUMPTION, WHERE THE SKELETON IS SPENT. THREE REPEATS; MIN–MAX IN PARENTHESES

Configuration	Cross-chapter	Never-decl.	Score	Tier 1 input	Cost
MARD (full)	0.917 (0.895–0.930)	2 (0–4)	0.621	84,811	\$0.578
A1s, skeleton suppressed	0.898 (0.874–0.942)	0 (0–0)	0.730	28,147	\$0.498
A1f, findings suppressed	0.014 (0.000–0.021)	57 (49–67)	0.623	68,584	\$0.524

TABLE XI

NEGATIVE CONTROL. MARD AND B1 ON THE STRUCTURE-ABLATED PRIMARY DOCUMENT, THREE REPEATS EACH. A NULL DIFFERENCE WAS PREDICTED; THE OBSERVED OUTCOME IS NOT A NULL

System	Output	Input tok.	Cost
B1, vanilla RLM	14,134	532,694	\$0.796
	13,563	225,268	\$0.465
	47,810	1,056,353	\$0.657
	words		
MARD (two-pass)	none	0	\$0.000
	(declined)	0	\$0.000
		0	\$0.000

is suppressed, linkage collapses to 1.4% and the compiler rejects a mean of 57 prerequisite declarations naming concepts that were never declared. Where structure is absent altogether, MARD produces nothing at all, at no cost.

This is the graceful-degeneration behaviour named in Section 1 as the intended failure mode, and we note that it was specified before it was observed. Its value is not that MARD scores well on a document it cannot process—it scores nothing—but that the failure is loud, immediate, free, and localised to a single named stage.

4.8. Structure as a faithfulness mechanism

A typed, validated plan makes every generated span carry a checkable claim about its own provenance, and makes the absence of usable structure a halt rather than a silent substitution. Those are the two properties the architecture delivers, and the negative control shows both at once.

They are narrower than the property we set out to claim, and Section 4.4 reports why: a validated plan does not make generated text *derive* from the document. MARD’s builders never see the document’s prose and its text-grounded count is zero.

Both halves matter, and the negative control separates them cleanly. Given a document whose structure has been destroyed, the baseline produced three complete study guides that named the intact book’s opening section and acknowledged nothing (Section 4.6); MARD produced nothing and said so, at zero cost, in under a tenth of a second. Neither system read the scrambled text usefully. Only one of them reported that.

4.9. Threats to validity

4.9.1. The comparison rests on one mechanism: If the envelope ceases to propagate downward, MARD is vanilla

RLM and every number in Table VI measures nothing—while every component still executes and no test fails. This mechanism is therefore guarded by dedicated tests asserting that a later chapter’s prompt contains an earlier chapter’s findings, and that a child envelope carries its parent’s findings. We note that those tests establish the envelope is *present* in the prompt, not that the model *used* it; ablation A1 is the first direct test of influence.

4.9.2. No repeat was excluded: One baseline repeat exhibited the grounding failure described in Section 4.4, and produced markedly different output volume, cost and latency from the other two. It is reported. A run is treated as re-runnable only when the *protocol* failed to execute—a harness fault, a provider fault, an unverified corpus. A run in which the system under test behaved badly is a result. Excluding the repeat in which the baseline performed worst would be indistinguishable from tuning toward a positive outcome.

4.9.3. Structure leakage: Section 3.2.5 describes the two routes by which document structure could reach a model without being derived. We control both and apply the controls to both arms. We cannot rule out a third route we have not thought of.

4.9.4. Single primary document: Manuscript-scale evidence from one document cannot establish that an effect is not document-specific. This is why variance across repeats, a negative control, and a pre-stated boundary carry the argument here rather than breadth, and why the full study expands the corpus.

4.9.5. Provider dependence: Results are obtained against a hosted inference endpoint whose decoding is not deterministic and whose behaviour may change. We mitigate by reporting repeats with spread and by recording a full configuration snapshot per run.

5. LIMITATIONS AND FUTURE WORK

5.1. Limitations

The system evaluated here is two-pass: the targeted deep-dive pass is specified but not executed, for the reasons given in Section 3.1.3. Evaluated output is restricted to explanations, and the task score is computed against learning-objective coverage and in-text cross-references only, with the glossary source deferred (Section 3.2).

Our vanilla-RLM baseline is our own implementation of the published method rather than the original authors’ code, adopted so that both arms share one implementation and

the comparison isolates the envelope (Section 3.2). Runs are additionally executed under bounded sub-call concurrency and input size imposed by our provider rate-limit tier; these bounds apply to both arms, but they place the absolute scores below what an unthrottled configuration would produce, and we therefore treat the *difference* between arms as the reportable quantity and the absolute values as configuration-specific.

The flat-context control is a structure-ablated variant of our own primary document rather than an external benchmark. That choice buys a clean manipulation—content held constant, structure removed—at the cost of not demonstrating degeneration on an independently constructed flat corpus, which the full study supplies.

Whether improved structural ordering translates into learning gain is not addressed—that requires a controlled study with human participants, which we scope as future work and do not speculate about here.

5.2. Future Work

Four extensions are already scoped rather than speculative, and each is named elsewhere in this manuscript as deferred rather than abandoned. First, executing Pass 2, the targeted deep dive specified in Section 3.1.3 but not run here, once Pass 1’s reliability has been evidenced over more than a single freeze window. Second, ablations A3 and A4—reordering disabled, and the sweep over recursion depth—which the full study reports and this manuscript does not, together with the external flat-context benchmark subset that complements the structure-ablated control used here (Section 4.6); its instance count places it outside this manuscript’s wall-clock budget rather than outside its scope. Third, expanding the corpus beyond a single primary document, together with the dependency-ordering case study on *Linear Algebra Done Right*, whose absent learning objectives rule it out as a primary document but not as an ordering target. Fourth, and outside this project’s scope, a controlled study with human participants would be required to establish whether improved structural ordering produces learning gain; we state this as an open question and do not speculate about its outcome.

Recovering glossary terms as a fourth ground-truth source, which requires font-level extraction to segment an un delimited run-on list, is a fifth deferred item rather than an abandoned one.

A sixth concerns presentation rather than measurement. The Master Plan’s concept graph is currently evaluated as a scored artefact rather than rendered as one, and Fig. 1 illustrates what changes when it is. Deployed products that generate document-derived mind maps render containment: the structure a document already declares through its headings. A prerequisite graph affords a different interaction—entering the material at a concept and walking backwards through what it depends on, rather than downwards through what contains it. Whether that serves a learner better than a hierarchical view is an open question, and answering it requires the controlled study scoped above rather than an appeal to either rendering’s appearance.

Two concrete obstacles stand between the artefact this manuscript measures and the artefact a reader would use, and we name them because neither is speculative.

The first is grounding. Section 4.4 reports that no Tier 2 builder call carries the cited prose, so MARD’s explanatory text is generated from parametric knowledge under a correct citation. For a scored artefact this is a disclosed limitation; for material a reader would study from, it is the first thing to fix, because prose that is page-anchored but not page-derived is precisely the failure a citation invites a reader to stop checking for. The repair is to supply each builder the span its concept was assigned—a change of a few lines, which would make text-grounding measurable where it is currently structurally absent. We note that the concept graph itself does not share this defect: concepts, their source spans and the edges between them are derived by the Tier 1 scout from the chapter text it reads. The structural artefact is grounded; the prose hung from it is not.

The second is rendering. A Master Plan is a directed acyclic graph in which 76 of 116 edges cross a chapter boundary and a single concept may depend on several others—in the run of Fig. 1(c), one concept in Chapter 4 depends on four concepts in Chapter 1. A mind-map renderer assigns each node one parent. Projecting the plan onto such a renderer therefore discards the majority of the edges that carry the contribution, and does so silently. Presenting a prerequisite graph to a reader requires a view that admits multiple parents and cross-branch edges; adopting an existing containment renderer unchanged would preserve the appearance of the contribution while deleting its substance.

CODE AND DATA AVAILABILITY

The implementation, the ingestion pipeline, every logged run directory, the evaluation modules, and the scripts that regenerate each figure and table in this paper are available at <https://github.com/brickstercodes/MARD-capstone>, at commit c1185a1. The supplementary results logs referenced in Section 4 are the `docs/` directory of that repository. Run directories carry the full configuration snapshot, the per-call log and the cost accounting for every number reported here, so each table is recomputable without an API key. The parsed corpora are not redistributed: `corpus/SOURCES.json` records each source document’s URL, SHA-256 and retrieval date, and `scripts/fetch_corpus.sh` reproduces them from the publishers.

6. CONCLUSION

Recursive decomposition removes the context bound but does not, on its own, produce a dependable artefact. Our measurements locate the instability precisely: run the same baseline three times on the same book and its output organisation, its cost and its coverage all move by more than the difference any of our conditions produced, and on two of those three runs its defining mechanism—delegation to sub-models—did not execute at all while the run reported success. Threading an operative metadata envelope through the

recursion does not make the generated explanations better; it makes the process repeatable, its output attributable to spans in the source, and its failure on unusable input immediate rather than silent. The mechanism responsible is narrower than the architecture we proposed: the downward flow of accumulated findings carries the effect, and the structural skeleton, which dominates the frontier tier’s input budget, does not earn its cost on the document measured here. We regard the negative control as the clearest statement of what was gained. Given a book whose structure had been destroyed, one system stopped and the other wrote forty-eight thousand confident words. Knowing which of those a system will do, before it does it, is the property this work adds.

REFERENCES

- [1] A. L. Zhang, T. Kraska, and O. Khattab, “Recursive language models,” 2025.
- [2] K. Alizadeh, P. Shojaei, M. Cho, and M. Farajtabar, “Recursive language models meet uncertainty: The surprising effectiveness of self-reflective program search for long context,” arXiv:2603.15653, 2026.
- [3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, 2024, pp. 157–173.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proc. EMNLP*, 2023, pp. 13358–13376.
- [5] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” in *Findings of the ACL*, 2024, pp. 963–981.
- [6] Y. Li, B. Dong, F. Guerin, and C. Lin, “Compressing context to enhance inference efficiency of large language models,” in *Proc. EMNLP*, 2023, pp. 6342–6353.
- [7] Z. Li, Y. Liu, Y. Su, and N. Collier, “Prompt compression for large language models: A survey,” in *Proc. NAACL*, 2025, pp. 7182–7195.
- [8] L. Ou and M. Lapata, “Context-aware hierarchical merging for long document summarization,” in *Findings of the ACL*, 2025, pp. 5534–5561.
- [9] H. Kim and B. H. Kim, “NexusSum: Hierarchical LLM agents for long-form narrative summarization,” in *Proc. ACL*, 2025, pp. 10120–10157.
- [10] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs with preference data,” 2024.
- [11] Y. Moslem and J. D. Kelleher, “Dynamic model routing and cascading for efficient LLM inference: A survey,” 2026.
- [12] Z. Dong, H. Sharma, E. O’Toole, J. P. Champati, and K. Wu, “Pay for hints, not answers: LLM shepherding for cost-efficient inference,” 2026.
- [13] I. Li, V. Yan, T. Li, R. Qu, and D. Radev, “Unsupervised cross-domain prerequisite chain learning using variational graph autoencoders,” in *Proc. ACL-IJCNLP*, 2021, pp. 1005–1011.
- [14] J. Sun, Y. He, Y. Xu, J. Sun, and G. Sun, “A learning-path based supervised method for concept prerequisite relations extraction in educational data,” in *Proc. ACM CIKM*, 2024, pp. 2168–2177.
- [15] Y. Bai, Z. Liu, T. Guo, M. Hou, and K. Xiao, “Prerequisite relation learning: A survey and outlook,” *ACM Computing Surveys*, 2025, doi:10.1145/3733593.
- [16] Z. Chu, S. Wang, J. Xie, T. Zhu, Y. Yan, J. Ye, A. Zhong, X. Hu, J. Liang, P. S. Yu, and Q. Wen, “LLM agents for education: Advances and applications,” in *Findings of the ACL: EMNLP*, 2025, pp. 13782–13810.
- [17] S. Sweta, “Educational data mining techniques with modern approach,” in *Modern Approach to Educational Data Mining and Its Applications*, SpringerBriefs in Applied Sciences and Technology. Singapore: Springer, 2021, pp. 25–38.
- [18] E. Tufino, “NotebookLM as a Socratic physics tutor: Design and preliminary observations of a RAG-based tool,” *The Physics Educator*, 2025, doi:10.1142/S2661339525500143.